DATABASE MANAGEMENT SYSTEM (ENCS205)

LabReport Submittedto

K. R. Mangalam University

for

Bachelor of Technology in Computer Science and Engineering

Submitted by

Pratham Sharma (2401010229)

Course Teacher

MISS MANSI KAJAL

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

Sohna, Haryana 122103, India

Jan-2026

Table of Contents

Exp. No	Name of the Experiments	Page no	Date of Experiment	Date of Submission	Faculty Signature
1	To design an Entity Relationship (ER) diagram for the College database and convert the ER diagram into relational tables.	1			
2	To design an Entity Relationship (ER) diagram for the Company database and convert it into relational tables.	3			
3		5			
4		7			
5		10			
6		12			
7		14			
8		16			

Experiment No: 1

Aim:

To design an Entity Relationship (ER) diagram for the College database and convert the ER diagram into relational tables.

Description:

The system consists of entities such as Student, Class, Semester, Subject, and Marks. A student attends a class, enrolls in subjects, and scores marks in each subject. Each class belongs to a specific semester, and a semester can have multiple classes. The marks entity stores internal and final assessment details of students.This ER diagram helps in clearly defining relationships between entities and avoids data redundancy. It provides a strong foundation for designing a relational database for managing student academic records.

Entities and Attributes

1. STUDENT

   Representsastudent studying in the institution.

   Attributes:

   - USN (Primary Key)
   - SName Address
   - Phone

2. CLASS

Represents a class allotted to students.

Attributes:

- USN
- SSID (Section / Class ID)

3. SEMESTER

Representstheacademic semester.

Attributes:

- SSN
- Sem
- Sec

4. SUBJECT

Represents subjects taught in the course.

Attributes:

- Subcode (Primary Key)
- Title
- Credits

- Sem

---

5. MARKS

Represents the marks scored by students.

Attributes:

- USN
- SSD
- Test1
- Test2
- Final
- SCode

Relationships:

1. STUDENT – CLASS

Relationship: *Attends*

- A student attends a class.
  A class can have many students.
-

---

2. CLASS – SEMESTER

Relationship: *Belongs To*

- A class belongs to one semester.
- A semester can have multiple classes.

---

3. STUDENT – SUBJECT

Relationship: *Enrolls*

- A student can enroll in many subjects.
- A subject can have many students.

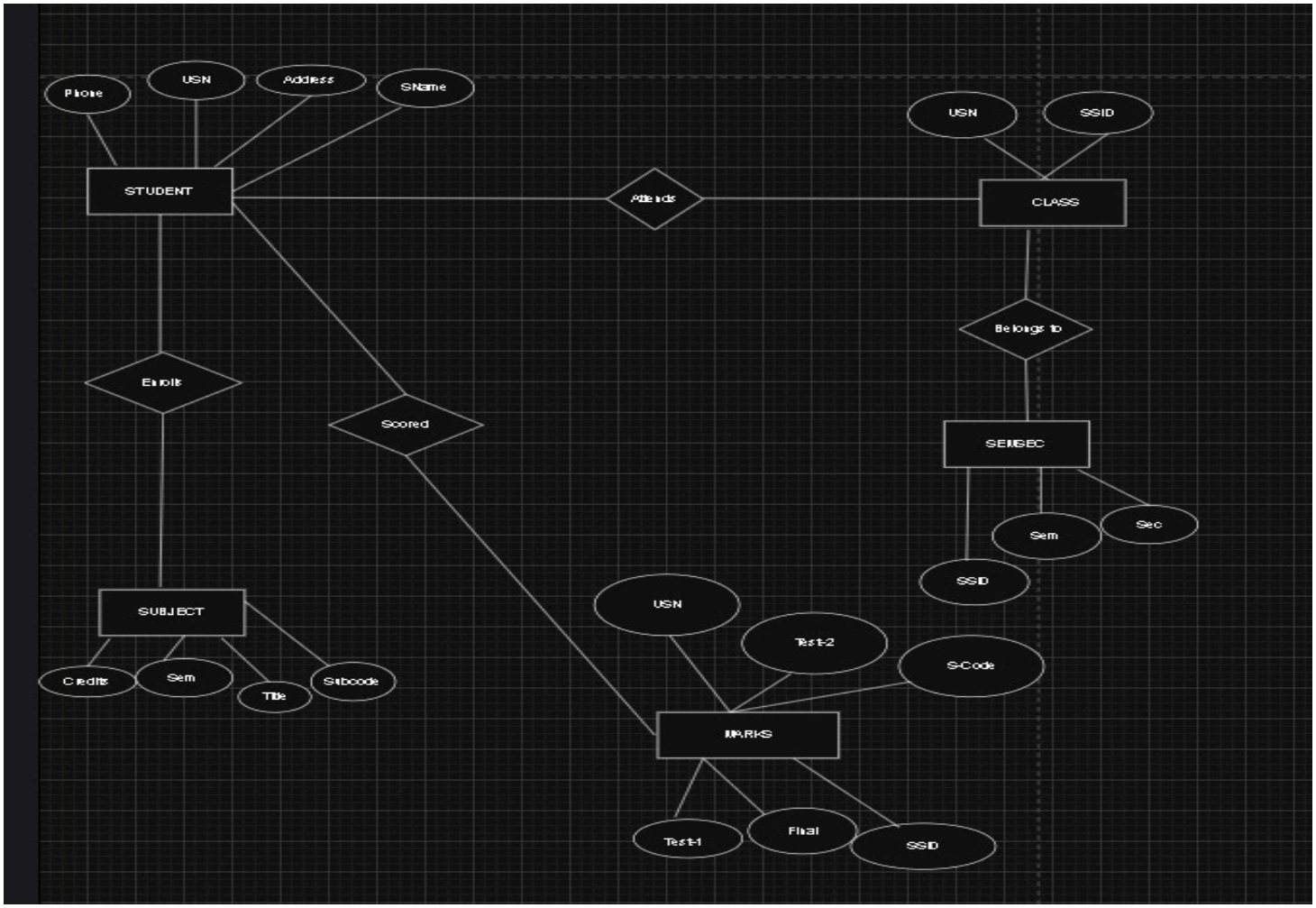
---

4. STUDENT – MARKS

Relationship: *Scored*

- A student scores marks in different subjects.
- Marks depend on the student and subject.

ER diagram:



Conclusion: This ER diagram clearly explains how students are relatedto classes, semesters, subjects, and marks. It helps in designing a proper database structure for managing academic records efficiently.

Experiment No.2

Aim:

Todesign an Entity Relationship (ER) diagram for the Company database and convert it into relational tables.

Description:

The system includes entities such as Employee, Department, Project, and Location. Employees work in departments, departments control projects, and employees may work on multiple projects. Departments can also be located in different locations. The ER diagram helps in organizing company data, reducing redundancy, and ensuring data integrity.

Entities and Attributes

EMPLOYEE

Represents an employee working in the organization.

Attributes:

- SSN (Primary Key)
- Name
- Address
- Salary
- Sex
- SuperSSN
- DNo

DEPARTMENT

Represents adepartment in the organization.

Attributes:

- DNo (Primary Key)
- DName
- MgrSSN
- MgrStartDate

PROJECT

Represents ap roject handled by the department.

A ttri butes**:**

- PNo (Primary Key)
- PName
- PLocation
- DNo

DLOCATION

Represents the location of a department.

Attributes:

- DLoc
- DNo

Relationships:

1. EMPLOYEE – DEPARTMENT
Relationship: *Worked*
An employee works in one department, and a department can have many employees.

---

2. DEPARTMENT – PROJECT
Relationship: *Works On*
A department controls many projects, and each project is associated with one department.

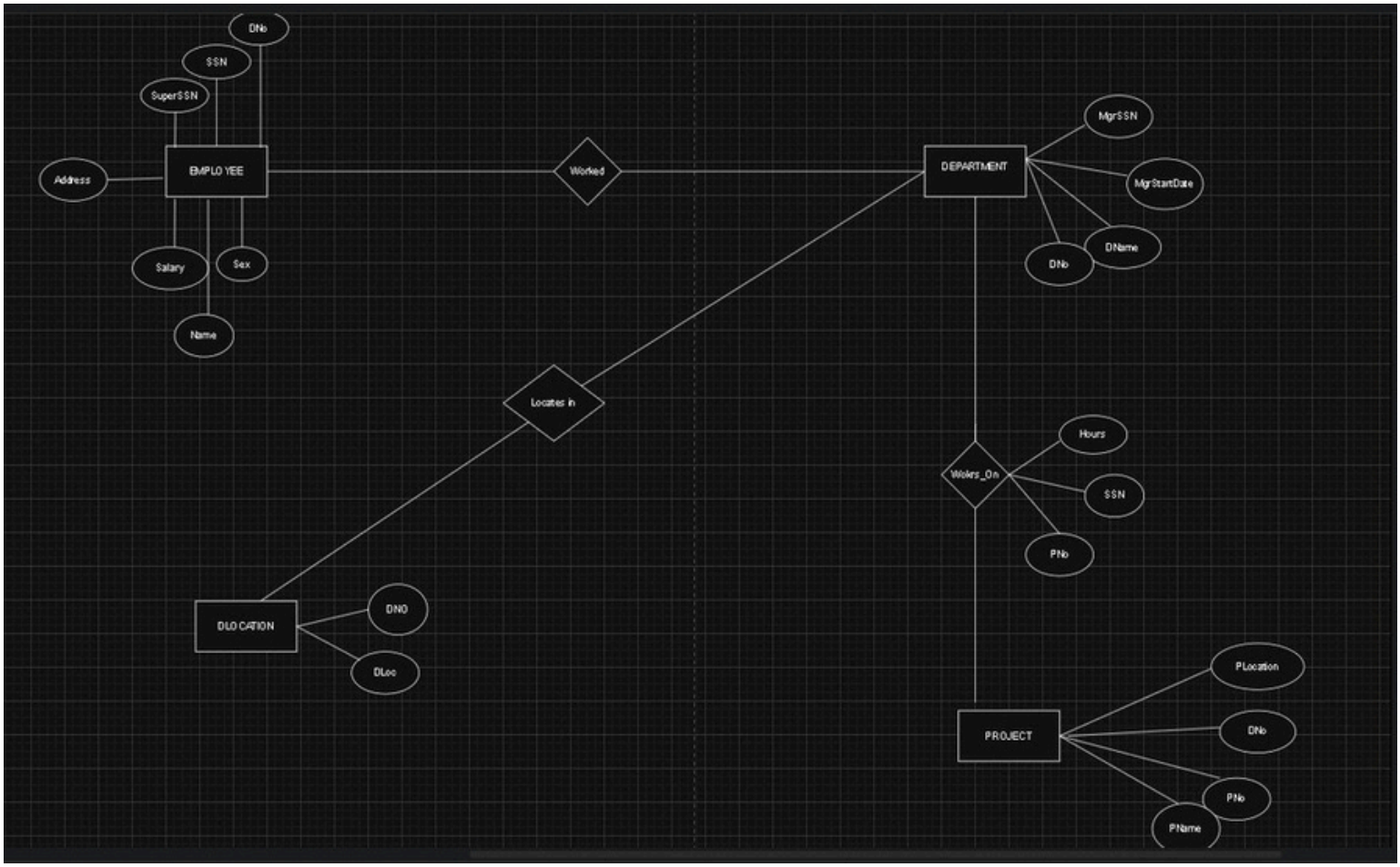
---

3. EMPLOYEE – PROJECT
Relationship: *Works On*
An employee can work on multiple projects, and a project can have multiple employees. The relationship stores the attribute **Hours**.

---

4. DEPARTMENT – LOCATION
Relationship: *Located In*
A department can be located in multiple locations.

ER diagram:



Conclusion:

ThisER diagramclearly illustrates the relationships between employees, departments, projects, and locations in a company. It serves as a strong foundation for designing a well-structured and efficient company database system.

ASSIGNMENT 3
PRATHAM SHARMA 2401010229

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following—

1. **Create the tables with the appropriate integrity constraints.**

```sql
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    PRIMARY KEY (bill_no, item_id),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2. Insert around 10 records in each of the tables.

```
INSERT INTO Item VALUES
(101,'Pen',20), (102,'Notebook',50), (103,'Bag',500),
(104,'Pencil',10), (105,'Shoes',1200),
(106,'Bottle',150), (107,'Watch',800),
(108,'Book',300), (109,'Headphones',700), (110,'Mouse',250);
```

```
INSERT INTO Customer VALUES
(1,'Amit'), (2,'Riya'), (3,'Rahul'), (4,'Sneha'), (5,'Karan'),
(6,'Neha'), (7,'Vikas'), (8,'Pooja'), (9,'Arjun'), (10,'Meera');
```

```
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,2),
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,105,1),
(3,CURDATE(),3,108,2),
(4,CURDATE(),4,102,5),
(5,CURDATE(),5,110,1),
(6,CURDATE(),6,107,1),
(7,CURDATE(),7,106,3),
(8,CURDATE(),8,104,10),
(9,CURDATE(),9,109,2);
```

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT s.bill_no, s.item_id, s.qty_sold, i.price,
       (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

6. Give a count of how many products have been bought by each customer

```
SELECT c.cust_id, c.cust_name, SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE VIEW Bill_View AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id,
       i.price, s.qty_sold,
       (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT bill_date,
       SUM(i.price * s.qty_sold) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
```

OUTPUT :

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	bill_no	cust_name	item_id
1	1	Ishaan	101
2	1	Ishaan	103
3	2	Mahin	105
4	3	Aditya	108
5	4	Ayush	102
6	5	Priya	110
7	6	Karan	107
8	7	Neha	106
9	8	Rohit	104
10	9	Simran	109

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	bill_no	item_id	qty_sold	price	amount
1	1	101	2	50000	100000
2	1001	101	1	50000	50000
3	4	102	5	500	2500
4	1002	102	2	500	1000
5	1	103	1	800	800
6	1007	103	2	800	1600
7	8	104	10	12000	120000
8	1003	104	1	12000	12000
9	2	105	1	200	200
10	1004	105	3	200	600
11	7	106	3	1500	4500
12	1009	106	1	1500	1500
13	6	107	1	900	900
14	1010	107	2	900	1800
15	3	108	2	25000	50000
16	1005	108	1	25000	25000
17	9	109	2	30000	60000
18	1008	109	1	30000	30000
19	5	110	1	700	700

Log | 1: Sale [10] | 2: Sale [20] | **3: Customer [10]** | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	cust_id	cust_name
1	1	Ishaan
2	4	Ayush
3	2	Mahin
4	7	Neha
5	8	Rohit
6	3	Aditya
7	9	Simran
8	6	Karan
9	10	Vikas
10	5	Priya

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | **4: Customer [10]** | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	cust_id	cust_name	total_products
1	1	Ishaan	4
2	2	Mahin	3
3	3	Aditya	3
4	4	Ayush	8
5	5	Priya	2
6	6	Karan	5
7	7	Neha	5
8	8	Rohit	11
9	9	Simran	3
10	10	Vikas	2

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | **5: Sale [2]** | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	item_name
1	Pen Drive
2	Tablet

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | **6: Item [10]**

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	item_id	item_name	price
1	101	Keyboard	50000
2	103	Keyboard	800
3	105	USB Cable	200
4	108	Tablet	25000
5	102	Mouse	500
6	110	Pen Drive	700
7	107	Charger	900
8	106	Headphones	1500
9	104	Monitor	12000
10	109	Smartphone	30000

QUESTION : Assignment 4
Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- **INSERT INTO class VALUES (5,’Rahul’);**
- **COMMIT;**
- **UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;**
- **SAVEPOINT A;**
- **INSERT INTO class VALUES (6, ‘Chris’);**
- **SAVEPOINT B;**
- **INSERT INTO class VALUES (7, ‘Bravo’);**
- **SAVEPOINT C**
- **SELECT * FROM class;**
- **ROLLBACK TO B;**
- **SELECT * FROM class;**
- **ROLLBACK TO A;**

CODE :

```
SET autocommit = 0;


CREATE TABLE class (
   id INT,
   name VARCHAR(50)
);



INSERT INTO class VALUES (1,'Bravo');
INSERT INTO class VALUES (2,'Alex');
INSERT INTO class VALUES (4,'Cheng');

INSERT INTO class VALUES (5,'Rahul');
COMMIT;

UPDATE class SET name='Abhijeet' WHERE id='5';

SAVEPOINT A;

INSERT INTO class VALUES (6,'Chris');
```

```
SAVEPOINT B;

INSERT INTO class VALUES (7,'Bravo');

SAVEPOINT C;

SELECT * FROM class;

ROLLBACK TO B;

SELECT * FROM class;

ROLLBACK TO A;
```

Table :

*	id	name
1	1	Bravo
2	2	Alex
3	4	Cheng
4	5	Abhijeet
5	6	Chris
6	7	Bravo

Assignment (4.2)

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

CODE :

```
CREATE TABLE ACCOUNT_TXN (
   Acc_ID INT,
   Holder_Name VARCHAR(50),
   Balance INT
);

INSERT INTO ACCOUNT_TXN VALUES (101,'Aarav',5000);
INSERT INTO ACCOUNT_TXN VALUES (102,'Bhavna',6000);
INSERT INTO ACCOUNT_TXN VALUES (103,'Chirag',7000);

SET autocommit = 0;

INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000);

UPDATE ACCOUNT_TXN SET Balance = Balance +1500 WHERE Acc_ID=101;

UPDATE ACCOUNT_TXN SET Balance = Balance -1000 WHERE Acc_ID=102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID=103;

INSERT INTO ACCOUNT_TXN VALUES (105,'Eshan',3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN
SET Holder_Name='Bhavna Sharma'
WHERE Acc_ID=102;


SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;
```

```
SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN
SET Balance = Balance +500
WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN;

SELECT * FROM ACCOUNT_TXN;

COMMIT;
```

OUTPUT :

*	Acc_ID	Holder_Name	Balance
1	101	Aarav	6500
2	102	Bhavna Sharma	5000
3	104	Diya	4000
4	105	Eshan	3000

Assignment 5
(Exercise on retrieving records from the table)
EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)
(a) Find out the employee id, names, salaries of all the employees
(b) List out the employees who works under manager 100
(c) Find the names of the employees who have a salary greater than or equal to 4800
(d) List out the employees whose last name is 'AUSTIN'
(e) Find the names of the employees who works in departments 60,70 and 80
(f) Display the unique Manager_Id.

CODE :

a)
```
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```

b)
```
SELECT *
FROM EMPLOYEES
WHERE Manager_Id = 100;
```

c)
```
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Salary >= 4800;
```

d)
```
SELECT *
FROM EMPLOYEES
WHERE Last_Name = 'AUSTIN';
```

e)
```
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Department_Id IN (60, 70, 80);
```

f)
```
SELECT DISTINCT Manager_Id
FROM EMPLOYEES;
```

Table :

*	Employee_Id	First_Name	Last_Name	Salary
1	101	John	Smith	5000.00
2	102	Sara	Austin	4800.00
3	103	Mike	Brown	6000.00
4	104	Lisa	Austin	4500.00
5	105	David	Lee	7000.00

Assignment 6
(Exercise on updating records in table)
Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due)
(a) Insert five records
(b) Find the names of clients whose bal_due> 5000 .
(c) Change the bal_due of ClientNO “ C123” to Rs. 5100
(d) Change the name of Client_master to Client12.
(e) Display the bal_due heading as “BALANCE”

CODE :
1. Create Database
```
CREATE DATABASE IF NOT EXISTS COMPANY_DB;
USE COMPANY_DB;
```

2. Create Table
```
CREATE TABLE Client_master (
   ClientNO VARCHAR(10) PRIMARY KEY,
   Name VARCHAR(50),
   Address VARCHAR(100),
   City VARCHAR(50),
   State VARCHAR(50),
   bal_due DECIMAL(10,2)
);
```

3. Insert Five Records
```
INSERT INTO Client_master VALUES
('C101', 'Rahul Sharma', 'MG Road', 'Delhi', 'Delhi', 6000),
('C102', 'Anita Verma', 'Park Street', 'Kolkata', 'West Bengal', 3000),
('C123', 'Raj Mehta', 'Link Road', 'Mumbai', 'Maharashtra', 4500),
('C104', 'Priya Singh', 'Banjara Hills', 'Hyderabad', 'Telangana', 7000),
('C105', 'Amit Patel', 'CG Road', 'Ahmedabad', 'Gujarat', 2000);
```

(b) Names of clients with bal_due > 5000
```
SELECT Name
FROM Client_master
WHERE bal_due > 5000;
```
(c) Update bal_due of ClientNO 'C123' to 5100
```
UPDATE Client_master
SET bal_due = 5100
WHERE ClientNO = 'C123';
```
Check update
```
SELECT * FROM Client_master WHERE ClientNO = 'C123';
```

(d) Rename table Client_master to Client12
```
ALTER TABLE Client_master
RENAME TO Client12;
```

(e) Display bal_due as "BALANCE"
```
SELECT ClientNO, Name, bal_due AS BALANCE
FROM Client12;
```

OUTPUT :

*	Acc_ID	Holder_Name	Balance
1	101	Aarav	6500
2	102	Bhavna Sharma	5000
3	104	Diya	4000
4	105	Eshan	3000

PRATHAM SHARMA 2401010229
Assignment 7
Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)
(a) Insert five records

```
INSERT INTO Teacher VALUES
('Amit', 101, '2020-06-15', 'Mathematics', 'Delhi', 40000),
('Riya', 102, '2019-03-10', 'Commerce', 'Mumbai', 35000),
('Rahul', 103, '2018-07-01', 'Science', 'Chennai', 45000),
('Sneha', 104, '2021-01-20', 'Mathematics', 'Delhi', 38000),
('Karan', 105, '2017-09-05', 'Commerce', 'Kolkata', 30000);
```

(b) Give Increment of 25% salary for Mathematics Department .

```
UPDATE Teacher
SET Salary = Salary + (Salary * 0.25)
WHERE DeptName = 'Mathematics';
```

(c) Perform Rollback command

```
ROLLBACK;
```

(d)GiveIncrementof15%salaryforCommerce Department

```
UPDATE Teacher
SET Salary = Salary + (Salary * 0.15)
WHERE DeptName = 'Commerce';
```

(e) Perform commit command

```
COMMIT;
```

Full code :

```
-- 1. CREATE DATABASE
CREATE DATABASE SchoolDB;
-- 2. USE DATABASE
USE SchoolDB;
-- 3. CREATE TABLE
CREATE TABLE Teacher (
  Name VARCHAR(50),
  DeptNo INT,
  Date_of_Joining DATE,
  DeptName VARCHAR(50),
  Location VARCHAR(50),
  Salary INT
);
-- 4. INSERT 5 RECORDS
INSERT INTO Teacher VALUES
('Amit', 101, '2020-06-15', 'Mathematics', 'Delhi', 40000),
('Riya', 102, '2019-03-10', 'Commerce', 'Mumbai', 35000),
('Rahul', 103, '2018-07-01', 'Science', 'Chennai', 45000),
('Sneha', 104, '2021-01-20', 'Mathematics', 'Delhi', 38000),
('Karan', 105, '2017-09-05', 'Commerce', 'Kolkata', 30000);
```

```sql
-- 5. START TRANSACTION

START TRANSACTION;

-- 6. (b) INCREMENT 25% SALARY FOR MATHEMATICS

UPDATE Teacher

SET Salary = Salary + (Salary * 0.25)

WHERE DeptName = 'Mathematics';

-- Check result

SELECT * FROM Teacher;

-- 7. (c) ROLLBACK (undo above change)

ROLLBACK;

-- Check again (salary should be original)

SELECT * FROM Teacher;

-- 8. START TRANSACTION AGAIN

START TRANSACTION;

-- 9. (d) INCREMENT 15% SALARY FOR COMMERCE

UPDATE Teacher

SET Salary = Salary + (Salary * 0.15)

WHERE DeptName = 'Commerce';

-- Check result

SELECT * FROM Teacher;

-- 10. (e) COMMIT (save permanently)

COMMIT;
```

```
-- Final check

SELECT * FROM Teacher;
```

OUTPUT :

Log | 1: Teacher [5]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	Name	DeptNo	Date_of_Joining	DeptName	Location	Salary
1	Amit	101	2020-06-15	Mathematics	Delhi	50000
2	Riya	102	2019-03-10	Commerce	Mumbai	35000
3	Rahul	103	2018-07-01	Science	Chennai	45000
4	Sneha	104	2021-01-20	Mathematics	Delhi	47500
5	Karan	105	2017-09-05	Commerce	Kolkata	30000

Pratham Sharma 2401010229
Assignment 8
(Exercise on order by and group by clauses) Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)
(a) Insert five records

```sql
INSERT INTO Sales VALUES
(1, 'Amit', 'Delhi', 5000, '2000-12-21'),
(2, 'Riya', 'Mumbai', 7000, '1999-05-15'),
(3, 'Rahul', 'Delhi', 6000, '2001-12-10'),
(4, 'Sneha', 'Chennai', 8000, '1998-07-25'),
(5, 'Karan', 'Mumbai', 5500, '2000-12-05');
```

(b) Calculate total salesamount in each branch

```sql
SELECT Branch, SUM(Salesamount) AS Total_Sales
FROM Sales
GROUP BY Branch;
```

(c) Calculate average salesamount in each branch

```sql
SELECT Branch, AVG(Salesamount) AS Avg_Sales
FROM Sales
GROUP BY Branch;
```

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09

```sql
SELECT Salesname,
       DATE_FORMAT(DOB, '%d-%b-%y') AS Formatted_DOB
FROM Sales
WHERE MONTH(DOB) = 12;
```

(e) Display the name and DOB of salesman in alphabetical order of the month.

```sql
SELECT Salesname, DOB
FROM Sales
ORDER BY MONTHNAME(DOB);
```

FULL CODE :

```
-- 1. CREATE DATABASE
CREATE DATABASE SalesDB;

-- 2. USE DATABASE
USE SalesDB;


-- 3. CREATE TABLE
CREATE TABLE Sales (
  SalesNo INT PRIMARY KEY,
  Salesname VARCHAR(50),
  Branch VARCHAR(50),
  Salesamount INT,
  DOB DATE
);


-- 4. INSERT 5 RECORDS
INSERT INTO Sales VALUES
(1, 'Amit', 'Delhi', 5000, '2000-12-21'),
(2, 'Riya', 'Mumbai', 7000, '1999-05-15'),
(3, 'Rahul', 'Delhi', 6000, '2001-12-10'),
(4, 'Sneha', 'Chennai', 8000, '1998-07-25'),
(5, 'Karan', 'Mumbai', 5500, '2000-12-05');


-- (b) TOTAL SALES AMOUNT IN EACH BRANCH
SELECT Branch, SUM(Salesamount) AS Total_Sales
FROM Sales
GROUP BY Branch;


-- (c) AVERAGE SALES AMOUNT IN EACH BRANCH
SELECT Branch, AVG(Salesamount) AS Avg_Sales
FROM Sales
GROUP BY Branch;


-- (d) SALESMEN BORN IN DECEMBER (FORMAT: 21-Dec-09)

SELECT Salesname,
     DATE_FORMAT(DOB, '%d-%b-%y') AS Formatted_DOB
FROM Sales
WHERE MONTH(DOB) = 12;
```

```
-- (e) DISPLAY NAME & DOB IN ALPHABETICAL ORDER OF MONTH

SELECT Salesname, DOB
FROM Sales
ORDER BY MONTHNAME(DOB);
```

OUTPUT :

Log | 1: Sales [3] | 2: Sales [3] | 4: Sales [5] | 3: Sales [3]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB

*	Branch	Total_Sales
1	Delhi	11000
2	Mumbai	12500
3	Chennai	8000

Log | 1: Sales [3] | 2: Sales [3] | 4: Sales [5] | 3: Sales [3]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB

*	Branch	Avg_Sales
1	Delhi	5500.0000
2	Mumbai	6250.0000
3	Chennai	8000.0000

Log | 1: Sales [3] | 2: Sales [3] | **4: Sales [5]** | 3: Sales [3]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BL

*	Salesname	DOB
1	Amit	2000-12-21
2	Rahul	2001-12-10
3	Karan	2000-12-05
4	Sneha	1998-07-25
5	Riya	1999-05-15

Log | 1: Sales [3] | 2: Sales [3] | 4: Sales [5] | **3: Sales [3]**

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB da

*	Salesname	Formatted_DOB
1	Amit	21-Dec-00
2	Rahul	10-Dec-01
3	Karan	05-Dec-00

Pratham Sharma 2401010229
Assignment 9
Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas
are as follows,
Departments (dept no , dept name , dept_location); Employees (emp_id , emp_name , emp_salary,dept_no);

a) Develop a query to grant all privileges of employees table into departments table

```
GRANT ALL PRIVILEGES
ON Employees
TO 'dept_user';
```

b) Develop a query to grant some privileges of employees table into departments table

```
GRANT SELECT, INSERT
ON Employees
TO 'dept_user';
```

c) Develop a query to revoke all privileges of employees table from departments table

```
REVOKE ALL PRIVILEGES
ON Employees
FROM 'dept_user';
```

d) Develop a query to revoke some privileges of employees table from departments table

```
REVOKE INSERT
ON Employees
FROM 'dept_user';
```

e) **Write a query to implement the save point.**

```
-- Start transaction
START TRANSACTION;

-- First operation
UPDATE Employees
SET emp_salary = emp_salary + 1000
WHERE dept_no = 1;

-- Create savepoint
SAVEPOINT sp1;

-- Second operation
UPDATE Employees
SET emp_salary = emp_salary + 2000
WHERE dept_no = 2;

-- Rollback to savepoint (undo second update only)
ROLLBACK TO sp1;

-- Commit final changes
COMMIT;
```

FULL CODE :

```
-- 1. CREATE DATABASE
CREATE DATABASE CompanyDB;

-- 2. USE DATABASE
USE CompanyDB;


-- 3. CREATE TABLES

CREATE TABLE Departments (
  dept_no INT PRIMARY KEY,
  dept_name VARCHAR(50),
  dept_location VARCHAR(50)
);

CREATE TABLE Employees (
```

```sql
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  emp_salary INT,
  dept_no INT,
  FOREIGN KEY (dept_no) REFERENCES Departments(dept_no)
);


-- 4. INSERT SAMPLE DATA

INSERT INTO Departments VALUES
(1, 'HR', 'Delhi'),
(2, 'Finance', 'Mumbai'),
(3, 'IT', 'Bangalore');

INSERT INTO Employees VALUES
(101, 'Amit', 30000, 1),
(102, 'Riya', 35000, 2),
(103, 'Rahul', 40000, 3),
(104, 'Sneha', 32000, 1),
(105, 'Karan', 37000, 2);


-- 5. CREATE USER (for privilege operations)
CREATE USER 'dept_user' IDENTIFIED BY '1234';


-- (a) GRANT ALL PRIVILEGES
GRANT ALL PRIVILEGES
ON Employees
TO 'dept_user';


-- (b) GRANT SOME PRIVILEGES
GRANT SELECT, INSERT
ON Employees
TO 'dept_user';


-- (c) REVOKE ALL PRIVILEGES
REVOKE ALL PRIVILEGES
ON Employees
FROM 'dept_user';


-- (d) REVOKE SOME PRIVILEGES
REVOKE INSERT
ON Employees
```

```sql
FROM 'dept_user';


-- (e) SAVEPOINT IMPLEMENTATION

START TRANSACTION;

UPDATE Employees
SET emp_salary = emp_salary + 1000
WHERE dept_no = 1;

SAVEPOINT sp1;

UPDATE Employees
SET emp_salary = emp_salary + 2000
WHERE dept_no = 2;

ROLLBACK TO sp1;

COMMIT;
CREATE USER 'dept_user'@'localhost' IDENTIFIED BY '1234';

-- FINAL CHECK
SELECT * FROM Employees;
```

OUTPUT :

Log | 1: Employees [5]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	emp_id	emp_name	emp_salary	dept_no
1	101	Amit	31000	1
2	102	Riya	35000	2
3	103	Rahul	40000	3
4	104	Sneha	33000	1
5	105	Karan	37000	2

PRATHAM SHARMA 2401010229

Assignment 10
Using the tables “DEPARTMENTS” and “EMPLOYEES” perform the following queries
a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT e.emp_id, e.emp_name, e.emp_salary,
       d.dept_name, d.dept_location
FROM Employees e
INNER JOIN Departments d
ON e.dept_no = d.dept_no;
```

b) Display the employee name and Department name by implementing a left outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_no = d.dept_no;
```

c) Display the employee name and Department name by implementing a right outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_no = d.dept_no;
```

d) Display the details of those who draw the salary greater than the average salary

```
SELECT *
FROM Employees
WHERE emp_salary > (
    SELECT AVG(emp_salary) FROM Employees
);
```

Full code ;

```
-- 1. CREATE DATABASE
CREATE DATABASE CompanyDB;

-- 2. USE DATABASE
USE CompanyDB;


-- 3. CREATE TABLES

CREATE TABLE Departments (
  dept_no INT PRIMARY KEY,
  dept_name VARCHAR(50),
  dept_location VARCHAR(50)
);

CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  emp_salary INT,
  dept_no INT,
  FOREIGN KEY (dept_no) REFERENCES Departments(dept_no)
);


-- 4. INSERT SAMPLE DATA

INSERT INTO Departments VALUES
(1, 'HR', 'Delhi'),
(2, 'Finance', 'Mumbai'),
(3, 'IT', 'Bangalore');

INSERT INTO Employees VALUES
(101, 'Amit', 30000, 1),
```

```
(102, 'Riya', 35000, 2),
(103, 'Rahul', 40000, 3),
(104, 'Sneha', 32000, 1),
(105, 'Karan', 37000, 2);


-- (a) INNER JOIN (Matching departments in both tables)

SELECT e.emp_id, e.emp_name, e.emp_salary,
       d.dept_name, d.dept_location
FROM Employees e
INNER JOIN Departments d
ON e.dept_no = d.dept_no;


-- (b) LEFT OUTER JOIN

SELECT e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_no = d.dept_no;


-- (c) RIGHT OUTER JOIN

SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_no = d.dept_no;


-- (d) EMPLOYEES WITH SALARY > AVERAGE SALARY

SELECT *
FROM Employees
WHERE emp_salary > (
  SELECT AVG(emp_salary) FROM Employees
);
```

OUTPUT :

Log | **1: Employees [5]** | 2: Employees [5] | 3: Employees [5] | 4: Employees [2]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	emp_id	emp_name	emp_salary	dept_name	dept_location
1	101	Amit	31000	HR	Delhi
2	104	Sneha	33000	HR	Delhi
3	102	Riya	35000	Finance	Mumbai
4	105	Karan	37000	Finance	Mumbai
5	103	Rahul	40000	IT	Bangalore

Log | 1: Employees [5] | **2: Employees [5]** | 3: Employees [5] | 4: Employees [2]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	emp_name	dept_name
1	Amit	HR
2	Riya	Finance
3	Rahul	IT
4	Sneha	HR
5	Karan	Finance

Log | 1: Employees [5] | 2: Employees [5] | **3: Employees [5]** | 4: Employees [2]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	emp_name	dept_name
1	Amit	HR
2	Sneha	HR
3	Riya	Finance
4	Karan	Finance
5	Rahul	IT

Log | 1: Employees [5] | 2: Employees [5] | 3: Employees [5] | **4: Employees [2]**

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	emp_id	emp_name	emp_salary	dept_no
1	103	Rahul	40000	3
2	105	Karan	37000	2